

# MLOps Pipeline — Heart Disease Prediction

---

SANDIP BHATTACHARYYA (2025cs05025) · 6th May 2026

[https://github.com/sandbh/ml\\_learning.git](https://github.com/sandbh/ml_learning.git)

## Table of Contents

1. Introduction, repository & deliverables
  2. Setup & Installation Instructions
  3. Data Acquisition & Exploratory Data Analysis
  4. Feature Engineering & Model Development
  5. Experiment Tracking with MLflow
  6. Model Packaging & Reproducibility
  7. CI/CD Pipeline & Automated Testing
  8. Model Containerization
  9. Production Deployment
  10. Monitoring & Logging
  11. Architecture (diagram)
  12. Evidence reference
-

## 1. Introduction

### 1.1 Problem & dataset

---

Heart disease is one of the leading causes of death globally. Early detection and risk assessment from routine clinical measurements can improve outcomes and prioritisation. This project builds an end-to-end Machine Learning Operations (MLOps) pipeline to predict heart-disease risk from the UCI Heart Disease dataset, covering the full lifecycle: data acquisition, exploratory analysis, model development, experiment tracking with MLflow, automated unit tests, continuous integration and delivery, containerisation, optional Kubernetes deployment, and production-style monitoring with Prometheus and Grafana—aligned with common industry MLOps practice.

The UCI Heart Disease dataset is used (see [data/](#)). After preprocessing, records include 13 numeric input features and a binary target (0 = no disease, 1 = disease). Feature groups include demographics (age, sex), clinical measures (cp, trestbps, chol, fbs, restecg), exercise-related variables (thalach, exang, oldpeak, slope), and angiographic indicators (ca, thal).

Data source: UCI Machine Learning Repository (Heart Disease).

### 1.2 Technology stack

---

| Component            | Technology                                                                 |
|----------------------|----------------------------------------------------------------------------|
| Programming language | Python 3.9+ (CI / Docker image: 3.11)                                      |
| ML framework         | scikit-learn (pinned in requirements.txt)                                  |
| API                  | FastAPI + Uvicorn (Flask in requirements is optional; main API is FastAPI) |
| Experiment tracking  | MLflow (local ./mlruns)                                                    |
| Testing              | pytest (tests/test_pipeline.py)                                            |
| CI/CD                | GitHub Actions (.github/workflows/ci.yml)                                  |
| Containerisation     | Docker + Docker Compose                                                    |
| Orchestration        | Kubernetes manifests under k8s/ (e.g. Minikube)                            |
| Monitoring           | Prometheus + Grafana (monitoring/)                                         |

### 1.3 Official repository

---

| Item        | Value                                                                                                          |
|-------------|----------------------------------------------------------------------------------------------------------------|
| URL         | <a href="https://github.com/sandbh/ml_learning.git">https://github.com/sandbh/ml_learning.git</a>              |
| Clone       | git clone<br><a href="https://github.com/sandbh/ml_learning.git">https://github.com/sandbh/ml_learning.git</a> |
| Root folder | ml_learning                                                                                                    |
| Author / ID | SANDIP BHATTACHARYYA — 2025cs05025                                                                             |

|          |                                                |
|----------|------------------------------------------------|
| Work dir | Repo root so data/, src/, models/ match README |
|----------|------------------------------------------------|

## 1.4 Deliverables map (repo paths)

| Required element                     | Location in this repository                                                                                                          |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Application & library code           | src/ (api/, config/, data_preprocessing/, eda/, model_training/, ...)                                                                |
| Dockerfile(s) & Compose              | Dockerfile at repo root; docker-compose.yml for API + Prometheus + Grafana                                                           |
| Python dependencies                  | requirements.txt (pinned scikit-learn for pickle compatibility)                                                                      |
| Conda environment file               | Not in repo. Use conda + pip install -r requirements.txt, or add env.yml if required                                                 |
| Raw & cleaned datasets               | data/heart_disease_UCI_dataset.csv (bundled raw);<br>data/heart_disease_processed_dataset.csv (generated by EDA preprocess)          |
| Dataset download / provenance        | README has the UCI link. Raw CSV is already in data/ (no download script)                                                            |
| EDA, training, inference             | Python modules: src/eda/eda.py, src/model_training/train.py, src/model_training/inference.py (no .ipynb in-tree; notebooks optional) |
| Unit / integration tests             | tests/ (pytest; pytest.ini sets testpaths=tests, pythonpath=src)                                                                     |
| CI/CD workflow                       | .github/workflows/ci.yml (GitHub Actions; no Jenkinsfile)                                                                            |
| Deployment manifests                 | k8s/ (Kustomize YAML). No Helm chart; you can add one if the course asks                                                             |
| Reporting screenshots                | screenshots/*.png (embedded in §12.1 when this report is regenerated)                                                                |
| Monitoring config                    | monitoring/prometheus.yml;<br>monitoring/grafana/ (dashboard JSON + provisioning)                                                    |
| Generated model artifacts (local/CI) | models/best_model.pkl,<br>models/feature_names.pkl,<br>models/training_metadata.json (recreated by train.py)                         |
| Experiment tracking store (local)    | ./mlruns/ for MLflow (often gitignored; CI can upload a zip artifact)                                                                |

## 2. Setup & Installation Instructions

This section gives a complete installation narrative: what must exist on the machine before you start, how to create an isolated Python environment, how dependencies are pinned for reproducible unpickling of the model, and which optional tools unlock Docker Compose and Kubernetes demos. For edge cases (port conflicts, Minikube drivers, Apple Silicon), the README troubleshooting section is the authoritative extension of these steps.

### 2.1 Prerequisites

- Python 3.9 or newer on the host (3.11 is used in CI and in the API Docker image). Verify with: `python3 --version`.
- Git, to clone and push the repository.
- A C compiler toolchain may be required on some platforms when pip builds binary wheels (gcc is also installed in the Dockerfile).
- Docker Desktop or an equivalent OCI runtime (optional) for docker build, docker compose, and image load into Minikube.
- kubectl and Minikube (optional) only if you demonstrate the k8s/ manifests.

### 2.2 Local setup

Create a dedicated virtual environment so that scikit-learn, MLflow, FastAPI, and pytest versions match those used during training and in CI. The project pins scikit-learn==1.6.1 because joblib pickles are not guaranteed compatible across arbitrary sklearn major/minor versions. After activation, upgrade pip, then install from requirements.txt. Full detail (ports, troubleshooting, K8s): README.md in the repository.

| Step | Command / action                                                                        | Notes                                                                                            |
|------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 1    | <code>git clone https://github.com/sandbh/ml_learning.git</code>                        | Then: <code>cd ml_learning</code>                                                                |
| 2    | <code>python3 -m venv .venv</code>                                                      | Activate: <code>source .venv/bin/activate</code> (Windows: <code>.venv\Scripts\activate</code> ) |
| 3    | <code>pip install -r requirements.txt</code>                                            | Pins scikit-learn for pickle compatibility                                                       |
| 4    | <code>python3 src/eda/eda.py all</code>                                                 | Writes cleaned CSV + screenshots/                                                                |
| 5    | <code>python3 src/model_training/train.py</code>                                        | GridSearch + MLflow; writes models/ and mlruns/                                                  |
| 6    | <code>python3 -m pytest tests/ -v</code>                                                | Unit tests                                                                                       |
| 7    | <code>python3 src/api/api.py</code>                                                     | Or: <code>uvicorn api:app --app-dir src --host 0.0.0.0 --port 8000</code>                        |
| 8    | <code>python3 -m mlflow ui --backend-store-uri ./mlruns --host 127.0.0.1 -p 5050</code> | Experiment UI                                                                                    |

#### Shell sequence (repository root):

```
git clone https://github.com/sandbh/ml_learning.git
cd ml_learning
python3 -m venv .venv
python3 -m pip install --upgrade pip
pip install -r requirements.txt
python3 src/eda/eda.py all
python3 src/model_training/train.py
python3 -m pytest tests/ -v
```

```
python3 src/api/api.py
# Swagger UI: http://127.0.0.1:8000/docs
```

| Verification | Expected                        |
|--------------|---------------------------------|
| After pytest | Imports and paths OK            |
| After train  | models/best_model.pkl + mlruns/ |

### 3. Data Acquisition & Exploratory Data Analysis

| Step    | What we did                                                   | Output / path                            |
|---------|---------------------------------------------------------------|------------------------------------------|
| Source  | UCI Heart Disease CSV in repo                                 | data/heart_disease_UCI_dataset.csv       |
| Clean   | pre_processing_data.py (missing tokens, types, binary target) | In-memory matrix; same code in train/API |
| EDA CLI | src/eda/eda.py preprocess   eda   all                         | data/heart_disease_processed_dataset.csv |
| Plots   | Histograms, correlation, class balance, agextarget            | screenshots/*.png (default names)        |
| Re-run  | Single command for full EDA + preprocess                      | python3 src/eda/eda.py all               |

#### 3.1 Exploratory Data Analysis

##### EDA — Key Insights:

- Age distribution: ranges from 29 to 77 years with a mean of ~54 years; slightly right-skewed.
- Gender: majority male (~68.3% male, ~31.7% female).
- Chest pain: type 0 is the most common (~47.2% of cases).
- Blood pressure: mostly between 120-140 mmHg.
- Fasting blood sugar: most patients below 120 mg/dl (85.1%).
- Heart disease prevalence: slightly more than half have heart disease (~54.5%).
- Correlations: age, exercise-induced angina, ST depression (oldpeak), and maximum heart rate show strong associations with the target.

## 4. Feature Engineering & Model Development

| Model               | Pipeline (sklearn)                                                                                           |
|---------------------|--------------------------------------------------------------------------------------------------------------|
| Logistic regression | SimpleImputer(median) → StandardScaler → LogisticRegression(liblinear, class_weight=balanced, max_iter=2000) |
| Random forest       | SimpleImputer(median) → RandomForestClassifier(class_weight=balanced); no scaler                             |

### 4.1 Data splits

| Stage / split        | Approx. fraction | Rows used for                                                | Implementation notes                                |
|----------------------|------------------|--------------------------------------------------------------|-----------------------------------------------------|
| Full cleaned dataset | 100%             | EDA plots, optional exploration                              | heart_disease_processed_dataset.csv                 |
| Training split       | ~80%             | GridSearchCV, refit, CV metrics, MLflow train metrics        | stratify=y, random_state=42                         |
| Hold-out test        | ~20%             | Final metrics, ROC/CM plots, model selection vs other family | Never inside CV folds                               |
| CV folds (×5)        | Training only    | Hyperparameter scoring (ROC-AUC)                             | StratifiedKFold, shuffle, random_state=42           |
| Inference / API      | N/A              | New patients or batch CSV                                    | No split; apply fitted Pipeline from best_model.pkl |

### 4.2 Hyperparameter search & evaluation

| Setting | Value                                                       |
|---------|-------------------------------------------------------------|
| Tuning  | GridSearchCV + StratifiedKFold(5), shuffle, random_state=42 |
| Scoring | roc_auc; refit on full train split                          |
| Winner  | Highest hold-out test ROC-AUC →                             |

|  |                       |
|--|-----------------------|
|  | models/best_model.pkl |
|--|-----------------------|

| Parameter            | Grid                 |
|----------------------|----------------------|
| lr__C                | 0.01, 0.1, 1.0, 10.0 |
| rf__n_estimators     | 100, 200             |
| rf__max_depth        | None, 6, 12          |
| rf__min_samples_leaf | 1, 2                 |

| Model               | Test metrics                               |
|---------------------|--------------------------------------------|
| Logistic Regression | roc_auc=0.9015, accuracy=0.8525, f1=0.8302 |
| Random Forest       | roc_auc=0.8755, accuracy=0.8033, f1=0.7778 |

| Training metadata       | Value                            |
|-------------------------|----------------------------------|
| Best model (metadata)   | Logistic Regression              |
| saved_at_utc (metadata) | 2026-05-06T15:05:15.015361+00:00 |
| CV / random_state       | cv_splits=5, random_state=42     |
| sklearn (metadata)      | 1.6.1                            |

| Plot artifact             | Path                                    |
|---------------------------|-----------------------------------------|
| CM — logistic regression  | screenshots/cm_logistic_regression.png  |
| CM — random forest        | screenshots/cm_random_forest.png        |
| ROC — logistic regression | screenshots/roc_logistic_regression.png |
| ROC — random forest       | screenshots/roc_random_forest.png       |

## 5. MLflow

| Topic      | Detail                          | Notes                               |
|------------|---------------------------------|-------------------------------------|
| Code       | src/model_training/train.py     | Logs each family after GridSearchCV |
| Store      | ./mlruns                        | ./mlruns (local file store)         |
| Experiment | heart_disease_prediction        | UI lists runs                       |
| Logged     | params, CV + test metrics, plot | Optional sklearn model log          |

|              |                                     |                                      |
|--------------|-------------------------------------|--------------------------------------|
|              | files                               |                                      |
| JSON summary | models/training_metadata.json       | Best model + metrics                 |
| UI tabs      | Parameters / Metrics /<br>Artifacts | Test ROC-AUC picks winner<br>in code |

#### MLflow UI command:

```
python3 -m mlflow ui --backend-store-uri ./mlruns --host 127.0.0.1 -p 5050
```

## 6. Artifacts & Batch Scoring

| File                          | Role               | Used by              |
|-------------------------------|--------------------|----------------------|
| models/best_model.pkl         | Fitted Pipeline    | inference.py, api.py |
| models/feature_names.pkl      | Column order       | api.py               |
| models/training_metadata.json | Metrics / versions | Report, audit        |

| Reproducibility | How                           |
|-----------------|-------------------------------|
| Versions        | requirements.txt sklearn pin  |
| Splits          | random_state=42 in train & CV |
| Cleaning        | Shared pre_processing_data.py |

#### Batch inference command:

```
python3 src/model_training/inference.py --output data/batch_predictions.csv
```

## 7. CI/CD Pipeline & Automated Testing

### 7.1 Automated tests

#### pytest configuration:

pytest.ini sets pythonpath=src and testpaths=tests. Tests under tests/ in tests/test\_pipeline.py (data load/clean, split checks, both sklearn pipelines, predict\_proba, PatientData schema).

#### Code block — run tests locally:

```
python3 -m pytest tests/ -v --tb=short
```



| CI job       | Purpose                        |
|--------------|--------------------------------|
| flake8       | Lint src/, tests/              |
| pytest       | JUnit + log artifacts          |
| train        | EDA + train.py → artifact zip  |
| docker smoke | Build image; /health, /predict |

### ci.yml excerpt:

```
# .github/workflows/ci.yml - CI/CD for MLOps assignment (course S2-
25_AMLCSZG523).
# Author: SANDIP BHATTACHARYYA - BITS Pilani ID 2025cs05025
#
# — When does this run?

```

---

```
#   • push / merge to branches: main, master only (other branches: open a PR)
#   • pull_request targeting main or master
#   • workflow_dispatch (Actions tab → CI/CD Pipeline → Run workflow)
#
# — Four jobs (sequential)

```

---

```
#   1. lint - flake8
#   2. testing - pytest + upload test-results (pytest-results.xml)
#   3. training-model - preprocess → train.py → upload ml-training (models/ +
mlruns/)
#   4. docker-build-smoke - needs training-model; docker build + /health +
/predict smoke
#
#   Later jobs show "Queued" until the previous job finishes. That is normal.
#
# — How long? (ballpark)

```

---

```
#   Often ~15-35 minutes total. Slow parts: pip install (per job),
GridSearchCV,
#   Docker build. training-model: 45m; lint/testing: 15m each.
#
name: CI/CD Pipeline

on:
  push:
    branches: [main, master]
  pull_request:
    branches: [main, master]
  workflow_dispatch:
```

| Screenshot (save to screenshots/) | Shows                                  |
|-----------------------------------|----------------------------------------|
| Actions list                      | Green workflow run                     |
| Job graph                         | lint → test → train → docker           |
| Logs                              | pytest tail, train upload, docker curl |
| Artifacts                         | XML, training zip, smoke logs          |
| Compose / k8s                     | Services or pods up                    |

## 8. Container (API)

| Image  | Detail                                   |
|--------|------------------------------------------|
| Base   | python:3.11-slim                         |
| App    | PYTHONPATH=/app/src; Uvicorn api.api:app |
| Health | GET /health                              |

### Dockerfile:

```
# Dockerfile – production-style image for the FastAPI inference API
# (assignment Task 6).
# Author: SANDIP BHATTACHARYYA – BITS Pilani ID 2025cs05025
#
# Expects trained artifacts under models/ at build time (best_model.pkl,
# feature_names.pkl).
# Build:  docker build -t heart-disease-api:latest .
# Run:    docker run --rm -p 8000:8000 heart-disease-api:latest
# Stack:  prefer docker-compose.yml for API + Prometheus + Grafana.
#
# — Stage 1: base image
```

---

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
# PYTHONPATH so ``api.api`` resolves to ``src/api/api.py``
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONPATH=/app/src \
    MODEL_PATH=/app/models/best_model.pkl \
    FEATURE_PATH=/app/models/feature_names.pkl \
    API_LOG_PATH=/app/logs/api_requests.log \
    PIP_ROOT_USER_ACTION=ignore
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
```

```
curl \
&& rm -rf /var/lib/apt/lists/* \
&& mkdir -p /app/logs
```

```
COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip \
    && pip install --no-cache-dir -r requirements.txt
```

```
COPY src/ ./src/
COPY models/ ./models/
```

```
EXPOSE 8000
```

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=20s --retries=3 \
    CMD curl -fsS http://127.0.0.1:8000/health >/dev/null || exit 1
```

### Build & run (host shell):

```
docker build -t heart-disease-api:latest .
docker run --rm -p 8000:8000 heart-disease-api:latest
```

## 8.1 API surface (FastAPI)

| Endpoint | Method | Description                                           |
|----------|--------|-------------------------------------------------------|
| /health  | GET    | Liveness; pipeline loaded                             |
| /predict | POST   | JSON body → prediction, label, confidence, risk_level |
| /metrics | GET    | Prometheus text format                                |
| /docs    | GET    | Swagger UI                                            |

### Smoke checks (API on port 8000):

```
curl -sS http://127.0.0.1:8000/health
curl -sS -X POST http://127.0.0.1:8000/predict \
    -H "Content-Type: application/json" \
    -d
'{"age":55,"sex":1,"cp":2,"trestbps":130,"chol":250,"fbs":0,"restecg":0,"thalach":150,"exang":0,"oldpeak":1.0,"slope":1,"ca":0,"thal":2}'
```

## 9. Deployment

| Compose service | URL                   | Role             |
|-----------------|-----------------------|------------------|
| API             | http://127.0.0.1:8000 | FastAPI          |
| Prometheus      | http://127.0.0.1:9090 | Scrapes /metrics |

|         |                       |                            |
|---------|-----------------------|----------------------------|
| Grafana | http://127.0.0.1:3000 | Dashboards (README: creds) |
|---------|-----------------------|----------------------------|

| Command / path                            | Purpose                                                                                                                                                                                                                                                           |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>docker compose up -d --build</code> | Stack: API + Prometheus + Grafana.<br>Refer README for more details<br><a href="https://github.com/sandbh/ml_learning#8-docker-api-image-only">https://github.com/sandbh/ml_learning#8-docker-api-image-only</a>                                                  |
| <code>kubectl apply -k k8s/</code>        | Optional K8s (Kustomize); See README for Minikube image load, namespaces, and port-forwards.<br>refer:<br><a href="https://github.com/sandbh/ml_learning#kubernetes-deployment-minikube">https://github.com/sandbh/ml_learning#kubernetes-deployment-minikube</a> |

## 10. Monitoring

| Piece         | Location                                                                     | Role                                                                 |
|---------------|------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Logs          | <code>src/api/api.py</code> ; <code>API_LOG_PATH</code> → <code>logs/</code> | Structured request log                                               |
| Metrics       | <code>GET /metrics</code> on API                                             | Counters, latency, predictions<br>( <code>prometheus_client</code> ) |
| Dashboard     | <code>monitoring/grafana/heart_disease_api.json</code>                       | Grafana panels                                                       |
| Scrape config | <code>monitoring/prometheus.yml</code>                                       | Compose scrape target                                                |

### prometheus.yml excerpt:

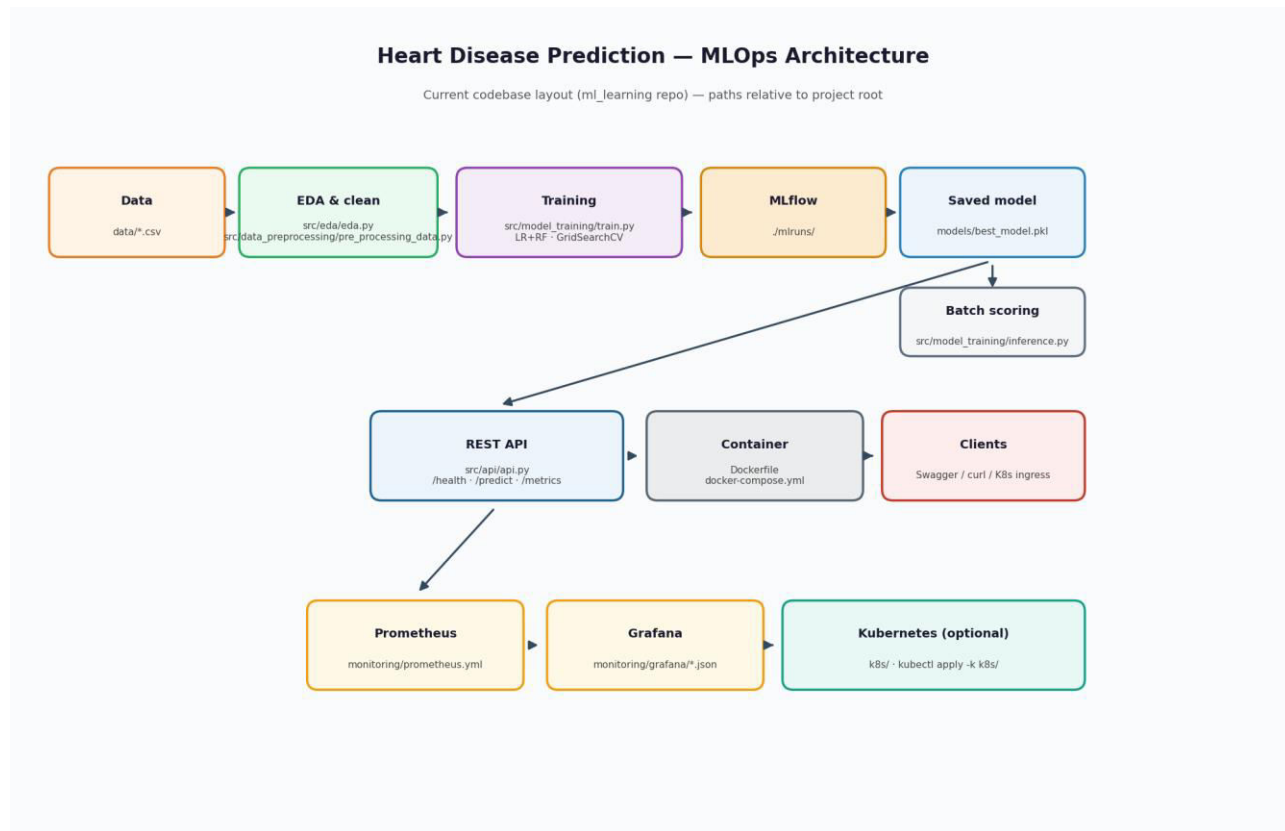
```
# Prometheus scrape config for Docker Compose (service name heart-disease-api).
# Mounted into the prometheus container via docker-compose.yml.
# Author: SANDIP BHATTACHARYYA — BITS Pilani ID 2025cs05025

global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  # Faster scrape during local Compose tests so counters appear in TSDB
  # quickly after /predict.
  - job_name: heart-disease-api
    scrape_interval: 5s
    # Prometheus requires scrape_timeout <= scrape_interval.
    scrape_timeout: 4s
```

```
metrics_path: /metrics
static_configs:
  - targets:
    - heart-disease-api:8000
```

## 11. Architecture



## 12. Evidence reference

Please refer:

1. Screenshots present in [https://github.com/sandbh/ml\\_learning/tree/main/screenshots](https://github.com/sandbh/ml_learning/tree/main/screenshots)
2. [https://github.com/sandbh/ml\\_learning/blob/main/reports/Heart\\_Disease\\_MLOps\\_Evidence\\_Sandip\\_Bhatta\\_charyya\\_2025cs05025.pdf](https://github.com/sandbh/ml_learning/blob/main/reports/Heart_Disease_MLOps_Evidence_Sandip_Bhatta_charyya_2025cs05025.pdf)